

37. Écrire une musique

Une mélodie qui joue toute seule, pendant que le jeu tourne.

1. Ce qu'il faut comprendre

Un `Air` s'écrit comme une tuile se dessine : une suite de pas, dans le programme. Chaque pas est une note et son volume — `"D04 12"` —, ou l'un des deux pas qui n'en sont pas : `"--"` fait taire la voix, `"=="` laisse la note d'avant continuer.

C'est `"=="` qui fait les notes longues. Sans lui, une blanche s'écrit en rejouant la même note à chaque pas, et l'oreille entendrait quatre coups au lieu d'une note tenue.

`jouer(voix, AIR, vitesse)` le lance. La **vitesse** est le nombre d'images que dure un pas : huit images font un peu moins de huit pas par seconde. Un quatrième argument, `1`, le fait recommencer sans fin.

Ensuite, **le programme n'a plus rien à tenir**. L'air avance tout seul, soixante fois par seconde, dans l'interruption de la console — même si le jeu, lui, est en retard. Un tempo qui ralentirait avec la charge du jeu ne serait pas un tempo.

La **partition ci-dessous** écrit ces lignes à la souris, et le bouton « Écouter » la fait entendre sans même compiler. Les deux voix chantantes sont la 1 et la 2 ; la quatrième, celle du bruit, se frappe avec `bruit(6, 10)`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1857 octets** de cartouche.

1. UNE MÉLODIE — THEME

```
Air THEME = {
    "D04 12", "==",    "MI4 12", "==",    "SOL4 12", "==",    "D05 13", "==",
    "SI4 11", "==",    "SOL4 11", "==",    "MI4 11", "==",    "--",    "--",
};
```

2. UNE MÉLODIE — BASSE

```
Air BASSE = {
    "D03 8",  "==",    "==",    "==",    "SOL3 8", "==",    "==",    "==",
    "MI3 8",  "==",    "==",    "==",    "--",    "--",    "--",    "--",
};
```

3. UNE VARIABLE — JOUE

```
uint8_t joue = 0;
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(4, 5, "LA MUSIQUE");
    texte(1, 8, "A  ARRETER OU JOUER");

    jouer(1, THEME, 8, 1);
    jouer(2, BASSE, 8, 1);
    joue = 1;

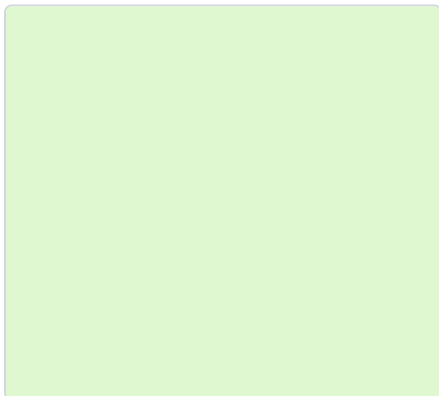
    while (true) {
        image();

        if (bouton(A) && joue) {
            silence(1);
            silence(2);
            joue = 0;
        }
    }

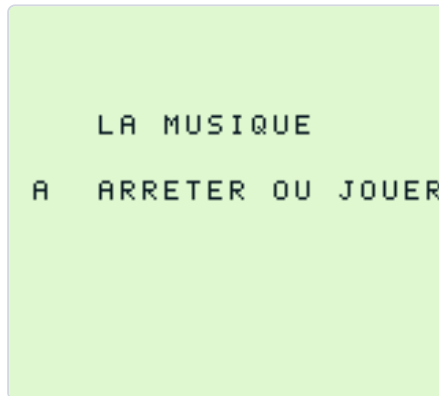
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

LA MUSIQUE
A ARRETER OU JOUER

A — l'écran n'a pas bougé

5. Ce qu'on doit voir

Rien de neuf à l'écran — mais deux voix qui jouent, et « A » qui les arrête.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le titre est écrit
- ✓ la voix 1 joue le DO4 du début (329, 392, 524, 495, 262 Hz)
- ✓ puis le MI4 et le SOL4 — l'air avance tout seul
- ✓ la voix 2 tient la basse (131, 196, 165 Hz)
- ✓ « A » fait taire les deux voix

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

THEME

BASSE

+ air

MIDI

WAV

THEME

Écouter

note

– silence

= tenir

▼

▲

– pas

+ pas

8

images par pas

DO5

DO4

vol

0

4

8

12

jouer(1, THEME, 8);

— le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas.

Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

Ensuite : **38. Une note, au front** — niveau 5 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++